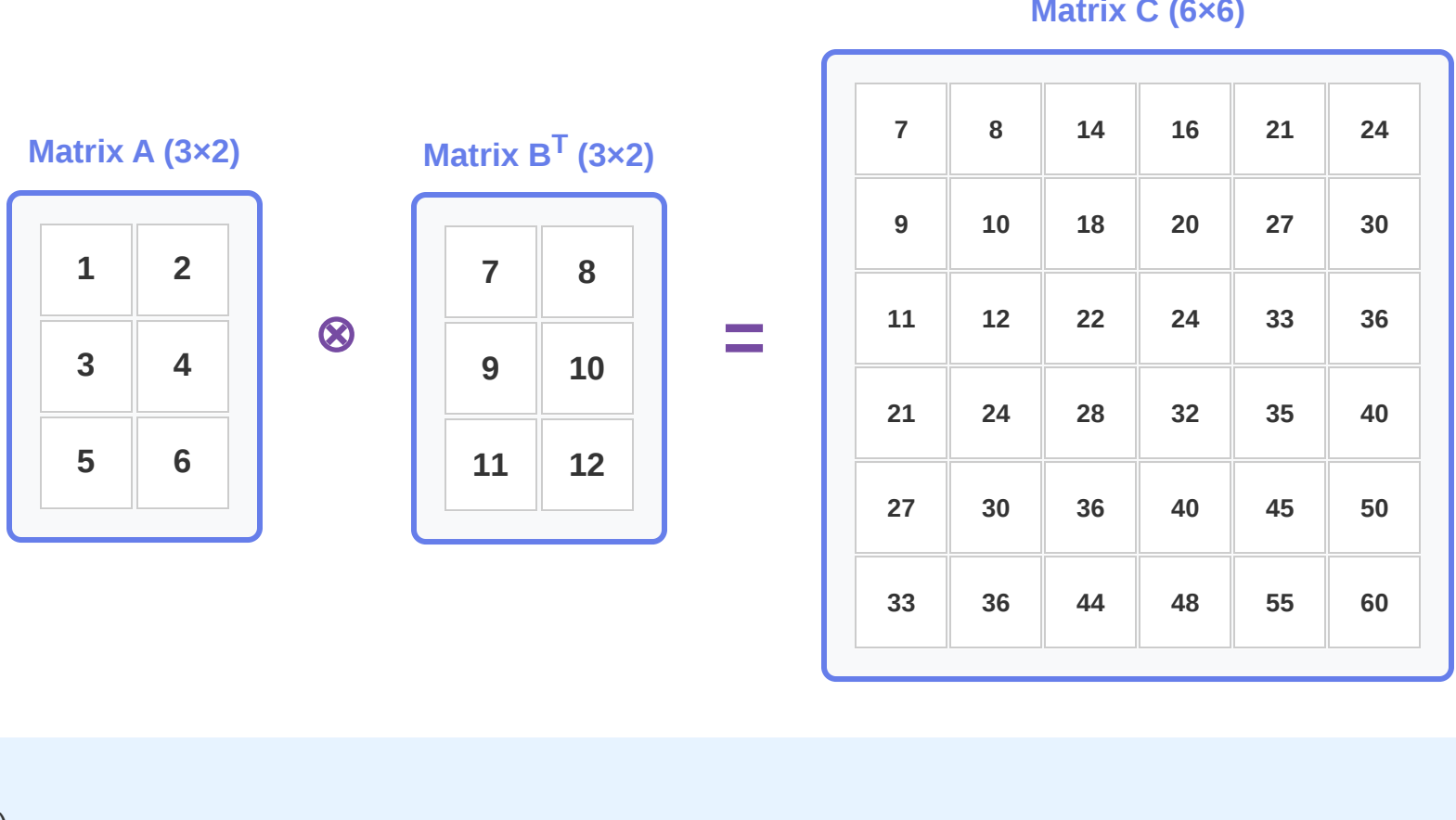


⊗ Kronecker Product: 3x2 Matrices Example

The Problem



- Problem Parameters:**
- m = 3 (number of rows)
 - n = 2 (number of columns)
 - Output size: m·n × m·n = 6 × 6 = 36 elements
 - Each input: 3×2 = 6 elements

9

Kernel 1 Threads

4

Kernel 2 Threads

36

Kernel 3 Threads

Kernel 1: Per-Row (9 Threads)

Strategy: Each thread handles one row from A and one row from B^T
Total Threads: m × m = 3 × 3 = 9 threads
Work per Thread: n × n = 2 × 2 = 4 output elements

Thread Organization:

Grid: 1D with 1 block (since 9 < 1024)
Block: 1D with 1024 threads (only 9 used)

Thread ID → (i, j) mapping:
i = thread_id / m // Row index in A
j = thread_id % m // Row index in B^T

Example: Thread 0 (i=0, j=0)

Thread 0

- Processes:**
- Row 0 of A: [1, 2]
 - Row 0 of B^T: [7, 8]

Computes 2x2 block in output:

```
for p=0 to 1: // Columns of A
  for q=0 to 1: // Columns of BT
    C[(2×0 + q)×6 + 2×p + 0] = A[0×2 + p] × B[0×2 + q]
```

```
C[0][0] = A[0][0] × B[0][0] = 1 × 7 = 7
C[0][2] = A[0][1] × B[0][0] = 2 × 7 = 14
C[1][0] = A[0][0] × B[0][1] = 1 × 8 = 8
C[1][2] = A[0][1] × B[0][1] = 2 × 8 = 16
```

All 9 Threads:

Thread ID	i (Row in A)	j (Row in B ^T)	Output Block	Elements Computed
0	0	0	C[0:2][0:2]	C[0][0], C[0][2], C[1][0], C[1][2]
1	0	1	C[0:2][3:5]	C[0][3], C[0][5], C[1][3], C[1][5]
2	0	2	C[0:2][6:8]	C[0][4], C[0][6], C[1][4], C[1][6]
3	1	0	C[2:4][0:2]	C[2][0], C[2][2], C[3][0], C[3][2]
4	1	1	C[2:4][3:5]	C[2][3], C[2][5], C[3][3], C[3][5]
5	1	2	C[2:4][6:8]	C[2][4], C[2][6], C[3][4], C[3][6]
6	2	0	C[4:6][0:2]	C[4][0], C[4][2], C[5][0], C[5][2]
7	2	1	C[4:6][3:5]	C[4][3], C[4][5], C[5][3], C[5][5]
8	2	2	C[4:6][6:8]	C[4][4], C[4][6], C[5][4], C[5][6]

Kernel 2: Per-Column (4 Threads)

Strategy: Each thread handles one column from A and one column from B^T
Total Threads: n × n = 2 × 2 = 4 threads
Work per Thread: m × m = 3 × 3 = 9 output elements

Thread Organization:

Grid: 1D with 1 block (since 4 < 1024)
Block: 2D with 32×32 threads (only 4 used)

Thread ID = (blockIdx.x × 32 × 32) + (threadIdx.y × 32) + threadIdx.x

Thread ID → (i, j) mapping:
i = thread_id / n // Column index in A
j = thread_id % n // Column index in B^T

Example: Thread 0 (i=0, j=0)

Thread 0

- Processes:**
- Column 0 of A: [1, 3, 5]
 - Column 0 of B^T: [7, 9, 11]

Computes 3x3 block in output:

```
for p=0 to 2: // Rows of A
  for q=0 to 2: // Rows of BT
    C[(2×p + 0)×6 + 2×q + 0] = A[p×2 + 0] × B[q×2 + 0]
```

```
C[0][0] = A[0][0] × B[0][0] = 1 × 7 = 7
C[0][1] = A[0][0] × B[1][0] = 1 × 9 = 9
C[0][2] = A[0][0] × B[2][0] = 1 × 11 = 11
C[2][0] = A[1][0] × B[0][0] = 3 × 7 = 21
C[2][1] = A[1][0] × B[1][0] = 3 × 9 = 27
C[2][2] = A[1][0] × B[2][0] = 3 × 11 = 33
C[4][0] = A[2][0] × B[0][0] = 5 × 7 = 35
C[4][1] = A[2][0] × B[1][0] = 5 × 9 = 45
C[4][2] = A[2][0] × B[2][0] = 5 × 11 = 55
```

All 4 Threads:

Thread ID	i (Col in A)	j (Col in B ^T)	Work
0	0	0	Col 0 of A × Col 0 of B ^T → 9 elements
1	0	1	Col 0 of A × Col 1 of B ^T → 9 elements
2	1	0	Col 1 of A × Col 0 of B ^T → 9 elements
3	1	1	Col 1 of A × Col 1 of B ^T → 9 elements

Kernel 3: Per-Element (36 Threads)

Strategy: Each thread computes ONE output element
Total Threads: m·n × m·n = 6 × 6 = 36 threads
Work per Thread: 1 multiplication (Maximum parallelism!)

Thread Organization:

Grid: 2D (gridDimx=1, gridDimy=1)
gridDimx = ceil(m·n / 16) = ceil(4/16) = 1
gridDimy = ceil(m·n / 64) = ceil(9/64) = 1
Block: 2D (64×16) = 1024 threads (only 36 used)

Thread ID = complex 2D grid/block calculation

Thread ID → (i, j) mapping:
i = thread_id / (m·n) // Row in output C
j = thread_id % (m·n) // Column in output C

Example Threads:

Thread 0

Computes C[0][0]
i=0, j=0
C[0][0] = A[0/2][0/2] × B[0%3][0%2]
= A[0][0] × B[0][0]
= 1 × 7 = 7

Thread 1

Computes C[0][1]
i=0, j=1
C[0][1] = A[0/2][1/2] × B[1%3][0%2]
= A[0][0] × B[1][0]
= 1 × 9 = 9

Thread 7

Computes C[1][1]
i=1, j=1
C[1][1] = A[1/2][1/2] × B[1%3][1%2]
= A[0][0] × B[1][1]
= 1 × 10 = 10

Thread 14

Computes C[2][2]
i=2, j=2
C[2][2] = A[2/2][2/2] × B[2%3][2%2]
= A[1][1] × B[2][0]
= 4 × 11 = 44

Thread 35

Computes C[5][5]
i=5, j=5
C[5][5] = A[5/2][5/2] × B[5%3][5%2]
= A[2][2] × B[2][1]
= 6 × 12 = 72

Index Mapping Formula:

$C[i][j] = A[i/n][j/m] \times B[j\%m][i\%n]$

- Where:
- i/n gives which "block row" (which row of A)
 - j/m gives which "block column" (which element within A's row)
 - j%*m* gives which row of B^T
 - i%*n* gives which column of B^T

Comparison Summary

Metric	Kernel 1 (Per-Row)	Kernel 2 (Per-Column)	Kernel 3 (Per-Element)
Total Threads	m ² = 9	n ² = 4	(m·n) ² = 36
Work per Thread	n ² = 4 multiplications	m ² = 9 multiplications	1 multiplication
Loops per Thread	2 nested loops (n×n)	2 nested loops (m×m)	No loops!
Grid Type	1D Grid	1D Grid	2D Grid
Block Type	1D Block (1024)	2D Block (32×32)	2D Block (64×16)
Thread Utilization	9/1024 = 0.9%	4/1024 = 0.4%	36/1024 = 3.5%
Parallelism Level	Low (9 parallel)	Very Low (4 parallel)	High (36 parallel)
Memory Access Pattern	Row-wise access	Column-wise access	Random access
Best Use Case	Large m, small n	Large n, small m	Balanced m and n, or very large matrices
Complexity	Simple indexing	Moderate indexing	Complex indexing (division/modulo)

Key Insights for 3x2 Example:

- Kernel 1: 9 Threads**
✓ Better than Kernel 2
✓ Each thread: 4 ops
✗ Only 0.9% GPU utilization

- Kernel 2: 4 Threads**
✗ Worst parallelism
✗ Each thread: 9 ops
✗ Only 0.4% GPU utilization

- Kernel 3: 36 Threads**
✓ Best parallelism!
✓ Each thread: 1 op
✓ 3.5% GPU utilization
✓ Scales best for large matrices

Performance Prediction:

9×

Speedup: Kernel 1 vs Sequential

4×

Speedup: Kernel 2 vs Sequential

36×

Speedup: Kernel 3 vs Sequential

Important Note:

For this small 3x2 example, all kernels underutilize the GPU (< 4% of 1024 threads per block). However, with larger matrices:

- **1000×1000 matrices:** Kernel 3 would launch 1,000,000 threads!
- **Kernel 3 scales best** because parallelism grows as O((m·n)²)
- **Kernel 1 & 2** have limited parallelism (O(m²) and O(n²))

This is why Kernel 3 (per-element) is typically preferred for large-scale GPU computing despite having more complex indexing.

Visual Representation of Thread Assignment

Output Matrix (6x6) with Thread Assignments:

Kernel 1 (Per-Row): Color = Thread ID

T0	T1	T2	T0	T1	T2
T0	T1	T2	T0	T1	T2
T3	T4	T5	T3	T4	T5
T3	T4	T5	T3	T4	T5
T6	T7	T8	T6	T7	T8
T6	T7	T8	T6	T7	T8

Each thread computes a 2x2 block (4 elements shown in same color)

Kernel 2 (Per-Column): Color = Thread ID

T0	T1	T2	T3	T0	T1
T0	T1	T2	T3	T0	T1
T0	T1	T2	T3	T0	T1
T2	T3	T2	T3	T2	T3
T2	T3	T2	T3	T2	T3
T2	T3	T2	T3	T2	T3

Each thread computes a 3x3 block scattered across the matrix (9 elements in same color)

Kernel 3 (Per-Element): Each cell = 1 unique thread

T0	T1	T2	T3	T4	T5
T6	T7	T8	T9	T10	T11
T12	T13	T14	T15	T16	T17
T18	T19	T20	T21	T22	T23
T24	T25	T26	T27	T28	T29
T30	T31	T32	T33	T34	T35

Perfect 1:1 mapping - Each thread computes exactly one element!